EurekAgent: Agent Environment Engineering is All You Need For Autonomous Scientific Discovery

Andrew Young
Automate Capture Research

June 13, 2026

Abstract

Autonomous scientific discovery demands computational agents that can explore hypothesis spaces while respecting resource constraints and maintaining reproducible provenance. We present `sandbox_science`, a Python library that provides a self-regulating, budget-aware execution sandbox, provenance-tracked Git collaboration, and mitigation of reward-hacking behaviours. The library follows a clean `src/` layout and exposes a concise import surface. Core components include a cost model, policy engine, execution sandbox, provenance auditor, and Git-based artifact manager. An exhaustive pytest suite validates three concrete properties: (i) enforcement of a user-specified computational budget, (ii) correct recording and retrieval of provenance information for generated artifacts, and (iii) detection of reward-hacking attempts. No empirical performance numbers are reported; the paper focuses on design, implementation, and qualitative validation through the test suite. Future work will evaluate the system on real scientific workloads.

1 Introduction

The rise of large language models and autonomous agents has opened the possibility of fully automated scientific discovery **??**. However, existing pipelines often neglect three critical aspects: (1) strict enforcement of computational budgets, (2) reproducible provenance of generated artifacts, and (3) robustness against agents that manipulate reward signals (reward hacking). Without these safeguards, autonomous agents can exhaust resources, produce irreproducible results, or converge on spurious objectives.

We propose **EurekAgent**, a software stack that treats the agent's execution environment as a first-class design variable. By engineering the sandbox, cost model, and provenance mechanisms together, we provide a principled foundation for safe, budget-aware autonomous discovery. The contribution of this work is a reference implementation, `sandbox_science`, that demonstrates the feasibility of this approach.

2 Related Work

Prior efforts have explored sandboxed execution for untrusted code **?**, cost-aware scheduling in cloud environments **?**, and provenance tracking in scientific workflows **?**. Reward-hacking mitigation has been studied in reinforcement learning safety **?**. Our work unifies these strands by providing a single library that integrates budget enforcement, Git-based provenance, and policy-driven reward monitoring for autonomous scientific agents.

3 Methodology

The architecture of `sandbox_science` is modular (Figure 1). The main components are:

- **cost_model**: Defines a deterministic mapping from primitive operations (e.g., CPU cycles, memory allocations) to abstract cost units. The model can be extended to incorporate external pricing APIs.
- **policy**: Implements a budget controller that monitors cumulative cost and aborts execution when the budget is exceeded. It also provides hooks for reward-hacking detection based on policy violations.
- **executor**: Wraps user code in a restricted Python environment, intercepting calls to the cost model and forwarding them to the policy engine.
- **sandbox**: Orchestrates the lifecycle of an execution, exposing a simple API `run_in_sandbox(func, budget)`.
- **provenance**: Records inputs, outputs, and environment snapshots in a Git repository. Each artifact is tagged with a unique hash and metadata linking it to the execution context.
- **auditor**: Analyzes provenance records to detect anomalies indicative of reward hacking (e.g., repeated generation of identical high-reward artifacts without corresponding cost).

Figure 1: Modular architecture of `sandbox_science`.

The execution flow is as follows: a user calls `sandbox.run_in_sandbox` with a target function and a budget. The executor creates a restricted namespace, instruments the function with cost

callbacks, and forwards cost events to the policy. Upon completion, the provenance module commits the generated artifacts to a Git repository, and the auditor scans the commit history for suspicious patterns.

4 Implementation

The library follows a conventional `src/` layout:

```
sandbox_science/
 src/
    sandbox_science/
        __init__.py
        auditor.py
        cost_model.py
        executor.py
        policy.py
        provenance.py
        sandbox.py
 tests/
     test_budget.py
     test_provenance.py
     test_reward_hacking.py
```

Each module provides a minimal public API:

- `sandbox_science.sandbox.run_in_sandbox(func, budget)`.
- `sandbox_science.cost_model.get_cost(op)`.
- `sandbox_science.policy.BudgetPolicy(budget)`.
- `sandbox_science.provenance.commit_artifact(path, metadata)`.
- `sandbox_science.auditor.detect_hacking(repo_path)`.

4.1 Testing Strategy

The test suite, written with `pytest`, contains three concrete acceptance tests:

1. **Budget-aware sandbox** (`test_budget.py`): Executes a synthetic workload whose cumulative cost exceeds a supplied budget. The test asserts that a `BudgetExceededError` is raised and that no further operations are performed.
2. **Artifact provenance** (`test_provenance.py`): Runs a function that produces a file, then checks that the file is committed to a temporary Git repository with correct metadata (hash, author, timestamp). Retrieval of the artifact via the provenance API is also verified.
3. **Reward-hacking detection** (`test_reward_hacking.py`): Simulates an agent that repeatedly returns a high-reward artifact without incurring cost. The auditor is invoked on the repository and the test asserts that the hacking flag is set.

All tests are deterministic, run in isolated temporary directories, and do not depend on external services. They collectively demonstrate that the library enforces budgets, tracks provenance, and flags reward-hacking behaviour.

5 Validation & Limitations

The test suite provides qualitative validation that the core design goals are met. Specifically, it shows that:

- The policy engine correctly aborts execution when the budget is exhausted.
- Provenance records are faithfully stored in Git and can be queried.
- The auditor can detect simple patterns of reward manipulation.

However, the implementation has not been evaluated on real scientific workloads, large-scale datasets, or in multi-agent settings. Performance overhead, scalability of the Git backend, and robustness to sophisticated hacking strategies remain open questions.

6 Conclusion and Future Work

We introduced `sandbox_science`, a reference Python library that integrates budget-aware sandboxing, Git-based provenance, and reward-hacking mitigation for autonomous scientific agents. The modular design and exhaustive test suite establish a solid foundation for further research. Future work will focus on empirical evaluation with real scientific pipelines, extending the cost model to cloud pricing APIs, improving the auditor with machine-learning-based anomaly detection, and exploring collaborative multi-agent scenarios.